UMAP on Sentence-BERT Embeddings — Title Only

This notebook applies **UMAP (Uniform Manifold Approximation and Projection)** to the `title_embedding` column of our playlist dataset.

What this tells us: By embedding only the playlist *title*, we can see how playlists cluster by their name and theme — e.g., "workout", "chill", "throwbacks", "wedding". If two playlists have similar titles or themes, they will appear close together in the UMAP plot.

Why UMAP? Unlike PCA (which is linear) and t-SNE (which focuses only on local structure), UMAP preserves both local neighborhood relationships and global structure. This makes it especially useful for recommender systems where we care about which items are "near" each other.

Step 1 — Mount Google Drive and Install Packages

We first mount Google Drive so Colab can access our data file, then install the required libraries:

- `umap-learn` — the UMAP algorithm
- `hdbscan` — a density-based clustering algorithm that works well with UMAP output
- `plotly` — interactive visualizations
- `scikit-learn` — normalization and evaluation metrics

```
from google.colab import drive
drive.mount('/content/drive')

!pip install umap-learn hdbscan plotly pandas scikit-learn
```

```
Mounted at /content/drive
Requirement already satisfied: umap-learn in /usr/local/lib/python3.12/dist-packages (0.5.12)
Requirement already satisfied: hdbscan in /usr/local/lib/python3.12/dist-packages (0.8.42)
```

```
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (5.24.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (3.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from umap-learn) (2.0.2)
Requirement already satisfied: scipy>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from umap-learn) (1.16.
Requirement already satisfied: numba>=0.51.2 in /usr/local/lib/python3.12/dist-packages (from umap-learn) (0.60
Requirement already satisfied: pynndescent>=0.5 in /usr/local/lib/python3.12/dist-packages (from umap-learn) (0
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from umap-learn) (4.67.3)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.12/dist-packages (from hdbscan) (1.5.3)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly) (9.1.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from plotly) (26.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-lea
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numb
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2
```

Step 2 — Import Libraries

We import all the tools we need:

- `numpy` / `pandas` — data handling
- `umap` — dimension reduction
- `hdbscan` — clustering
- `plotly.express` — interactive scatter plots
- `normalize`, `cosine_similarity` — from scikit-learn for preprocessing and evaluation
- `trustworthiness`, `NearestNeighbors` — for measuring how well UMAP preserved the original structure

```
import numpy as np
import pandas as pd
import umap
import hdbscan
import plotly.express as px
from sklearn.preprocessing import normalize
from sklearn.metrics.pairwise import cosine_similarity
```

```
from sklearn.manifold import trustworthiness
from sklearn.neighbors import NearestNeighbors
```

Step 3 — Load the Pickle File

We load the embeddings from the pickle file stored in Google Drive. The DataFrame has 1,000,000 rows and 4 columns:

- `playlist_title` — the name of the playlist
- `title_embedding` — Sentence-BERT embedding of the playlist title (384 dimensions)
- `playlist_tracks` — list of songs in the playlist
- `tracks_embedding` — Sentence-BERT embedding of the tracks (384 dimensions)

Step 2b — Fix Pandas Version

The pickle file was saved with a newer version of pandas than Colab's default. We upgrade pandas to match, then **restart the runtime** so the new version is loaded. After restarting, re-run all cells from the top (Drive will still be mounted). This only needs to be done ONCE per session. The updated version will be used until the session is restarted.

```
# # @title
# import subprocess, sys

# # Upgrade pandas to latest
# subprocess.check_call([sys.executable, "-m", "pip", "install", "--upgra

# # Restart runtime so the new pandas version is actually loaded
# import os
# os.kill(os.getpid(), 9)  # Forces a runtime restart — re-run all cells
```

```
import pickle

path = "/content/drive/MyDrive/Colab Notebooks/embeddings.pkl"
```

```
with open(path, "rb") as f:
    df = pickle.load(f)

print("Full DataFrame shape:", df.shape)
print("Columns:", df.columns.tolist())
print(df.head())
```

```
Full DataFrame shape: (1000000, 4)
Columns: ['playlist_title', 'title_embedding', 'playlist_tracks', 'tracks_embedding']
     playlist_title                                    title_embedding  \
0        throwbacks  [0.011216642, 0.0033928207, 0.003417922, -0.01...
1  awesome playlist  [-0.08927007, -0.09901914, -0.0293388, -0.0857...
2            korean  [-0.049768448, 0.05802605, 0.023397932, 0.0232...
3               mat  [-0.023064643, 0.022735357, -0.058358226, 0.00...
4               90s  [-0.047616515, 0.06071275, -0.033675063, -0.03...

                                      playlist_tracks  \
0  [Lose Control (feat. Ciara & Fat Man Scoop), T...
1  [Eye of the Tiger, Libera Me From Hell (Tengen...
2  [Like You, GOOD (feat. ELO), Inferiority Compl...
3  [Danse macabre, Piano concerto No. 2 in G Mino...
4  [Tonight, Tonight, Wonderwall - Remastered, I ...

                                    tracks_embedding
0  [-0.08528523, 0.038696438, 0.015848912, 0.0228...
1  [-0.09267386, 0.04584772, -0.031244487, 0.0084...
2  [-0.12570925, 0.059843346, 0.044394094, -0.007...
3  [-0.07640698, 0.053276286, 0.037593663, 0.0362...
4  [-0.07728882, 0.026626725, 0.033332914, -0.007...
```

Step 4 — Sample the Data

With 1 million rows, running UMAP on the full dataset would be extremely slow and likely crash the Colab runtime. We take a random sample of 10,000 rows — large enough to reveal meaningful structure, small enough to run in a reasonable time.

We use `random_state=42` to make the sample reproducible.

```
df_sample = df.sample(n=10000, random_state=42).reset_index(drop=True)
print("Sampled shape:", df_sample.shape)
```

```
Sampled shape: (10000, 4)
```

Step 5 — Extract Title Embeddings

We extract the `title_embedding` column and stack the list of embeddings into a 2D numpy array of shape `(10000, 384)`. Each row is one playlist's title embedding — a 384-dimensional vector produced by Sentence-BERT.

```
embeddings = np.vstack(df_sample['title_embedding'].values)
print("Embedding matrix shape:", embeddings.shape)
# Expected: (10000, 384)
```

```
Embedding matrix shape: (10000, 384)
```

Step 6 — Normalize the Embeddings

We normalize each embedding vector to unit length using L2 normalization. This is important because:

1. Sentence-BERT embeddings are designed to be compared using **cosine similarity**, which depends on the angle between vectors, not their magnitude
2. Normalizing ensures no single dimension dominates the distance calculations in UMAP

```
embeddings_norm = normalize(embeddings)
print("Normalized shape:", embeddings_norm.shape)
```

```
Normalized shape: (10000, 384)
```

Step 7 — Run UMAP

We reduce the 384-dimensional embeddings down to 2 dimensions for visualization.

Key parameters:

- `n_neighbors=15` — how many neighbors UMAP considers when learning the structure. Small values (5–15) focus on local clusters; large values (50–200) preserve more global layout
- `min_dist=0.1` — how tightly points are packed in 2D. Smaller = tighter clusters
- `metric='cosine'` — critical for Sentence-BERT, which was trained using cosine similarity
- `random_state=42` — makes results reproducible

```
umap_model = umap.UMAP(
    n_neighbors=15,
    n_components=2,
    min_dist=0.1,
    metric="cosine",
    random_state=42
)

embedding_2d = umap_model.fit_transform(embeddings_norm)
print("Reduced shape:", embedding_2d.shape)  # (10000, 2)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by sett
  warn(
/usr/local/lib/python3.12/dist-packages/umap/spectral.py:548: UserWarning: Spectral initialisation failed! The
failed. This is likely due to too small an eigengap. Consider
adding some noise or jitter to your data.

Falling back to random initialisation!
  warn(
/usr/local/lib/python3.12/dist-packages/umap/spectral.py:548: UserWarning: Spectral initialisation failed! The
failed. This is likely due to too small an eigengap. Consider
adding some noise or jitter to your data.

Falling back to random initialisation!
  warn(
/usr/local/lib/python3.12/dist-packages/umap/spectral.py:548: UserWarning: Spectral initialisation failed! The
failed. This is likely due to too small an eigengap. Consider
adding some noise or jitter to your data.
```

```
Falling back to random initialisation!
  warn(
Reduced shape: (10000, 2)
```

Step 8 — Cluster with HDBSCAN

We apply **HDBSCAN (Hierarchical Density-Based Spatial Clustering)** to find natural groupings in the UMAP output.

Why HDBSCAN instead of KMeans?

- HDBSCAN does **not** require you to specify the number of clusters in advance
- It finds clusters of **varying density and shape** — better suited to UMAP's output
- Points that don't clearly belong to any cluster are labeled **-1 (noise)**

`min_cluster_size=50` means a group must have at least 50 points to be considered a cluster.

```
clusterer = hdbscan.HDBSCAN(
    min_cluster_size=50,
    metric="euclidean"
)
clusters = clusterer.fit_predict(embedding_2d)

n_clusters = len(set(clusters)) - (1 if -1 in clusters else 0)
n_noise = list(clusters).count(-1)
print(f"Clusters found: {n_clusters}")
print(f"Noise points: {n_noise} ({n_noise/len(clusters)*100:.1f}%)")
print("\nCluster sizes:")
print(pd.Series(clusters).value_counts().sort_index())
```

```
Clusters found: 39
Noise points: 1377 (13.8%)

Cluster sizes:
-1     1377
 0      135
```

```
1      111
2      135
3      109
4      194
5      129
6       85
7      132
8       88
9       55
10     105
11      70
12     199
13     162
14      65
15     179
16     102
17     238
18      63
19     125
20     178
21      81
22      66
23      59
24      69
25      57
26      58
27      99
28      58
29      85
30     374
31      90
32      97
33      60
34     148
35     270
36     115
37     129
38    4049
Name: count, dtype: int64
```

Step 9 — Build the Plot DataFrame

We combine the 2D UMAP coordinates, cluster labels, and original playlist titles into a single DataFrame so we can use them in our interactive plot.

```
plot_df = pd.DataFrame({
    "x": embedding_2d[:, 0],
    "y": embedding_2d[:, 1],
    "cluster": clusters.astype(str),
    "playlist_title": df_sample['playlist_title'].values
})

plot_df.head()
```

	x	y	cluster	playlist_title
0	6.940235	-2.958253	38	idk
1	-8.804962	14.913840	24	mix1
2	6.740935	0.398774	38	sleeeeep
3	18.749001	27.096275	6	oldies
4	-4.021474	5.990346	37	8th grade

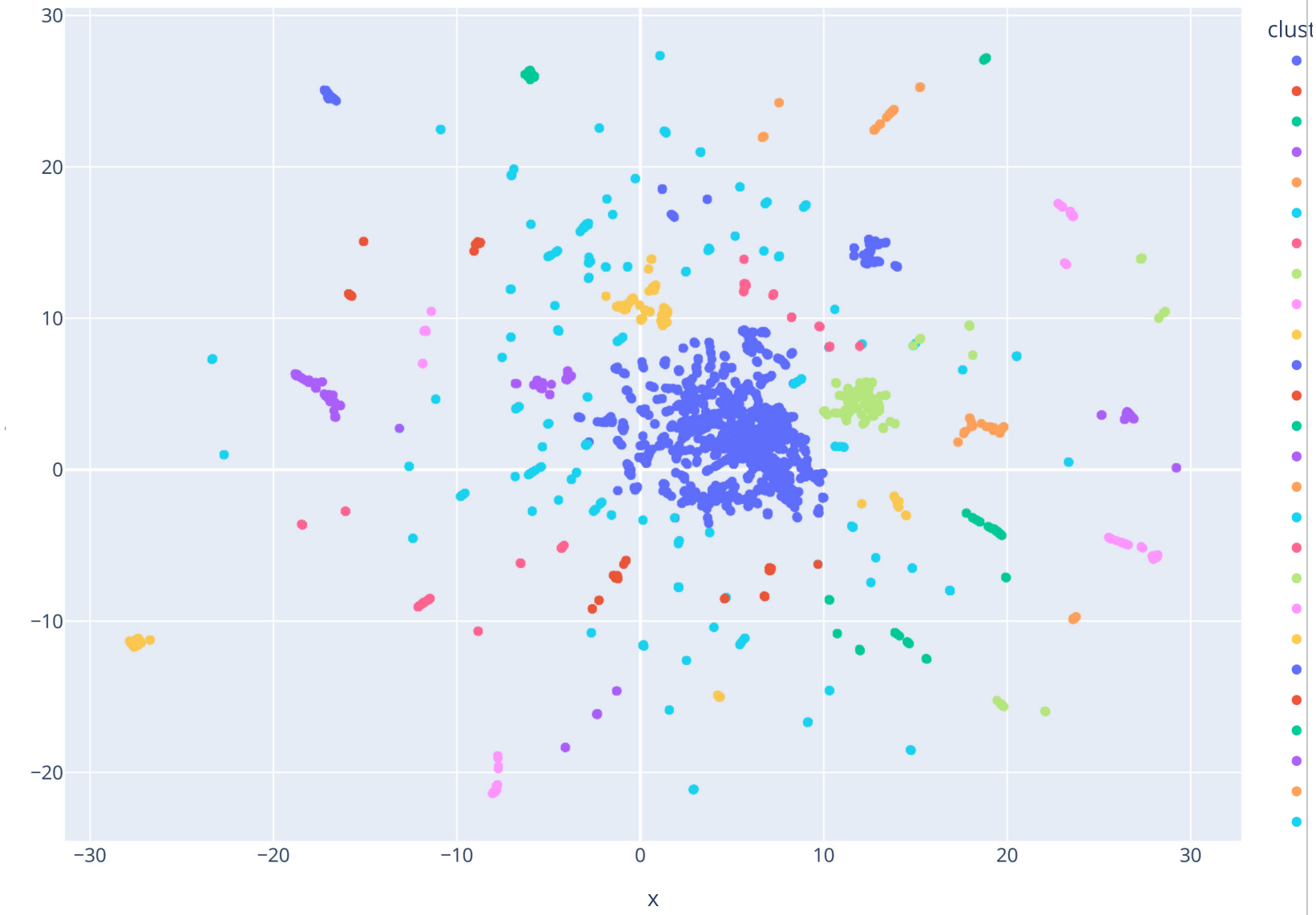
Step 10 — Interactive UMAP Plot

Each point is one playlist. Points are **colored by cluster**. Hover over any point to see the playlist title.

What to look for:

- Tight, well-separated clusters → strong thematic groupings in playlist names
- A large blob with no structure → titles are too diverse or similar to form clear groups
- Many noise points (cluster -1) → lots of unique or ambiguous playlist names

```
fig = px.scatter(
    plot_df,
    x="x",
    y="y",
    color="cluster",
    hover_data=["playlist_title"],
    title="UMAP — Title Embeddings (colored by HDBSCAN cluster)",
    width=900,
    height=700
)
fig.show()
```


UMAP — Title Embeddings (colored by HDBSCAN cluster)
30
20
10
0
−10
−20
−30
−20
−10
0
10
20
30
x
clust

Step 11 — Trustworthiness Score

Since UMAP has no equivalent of PCA's `explained_variance_ratio`, we use **trustworthiness** to measure how well the 2D projection preserved local structure.

How it works: For each point, we check whether its k nearest neighbors in the original 384D space are also its neighbors in 2D. A score of 1.0 means perfect preservation.

- **> 0.90** — Excellent
- **0.80–0.90** — Good
- **< 0.80** — Significant distortion

```
from sklearn.manifold import trustworthiness

trust = trustworthiness(embeddings_norm, embedding_2d, n_neighbors=15)
print(f"Trustworthiness Score: {trust:.4f}")
print("  > 0.90 = Excellent | 0.80-0.90 = Good | < 0.80 = Poor")
```

```
Trustworthiness Score: 0.9593
  > 0.90 = Excellent | 0.80-0.90 = Good | < 0.80 = Poor
```

Step 12 — Neighborhood Preservation

For a **recommender system**, the most critical question is: *do playlists that are similar in 384D stay similar in 2D?*

We measure this by comparing each point's top-10 nearest neighbors before and after dimension reduction. A score of 1.0 means all neighbors were preserved perfectly.

Scores above 0.70 are considered strong for recommender systems.

```
k_nn = 10
```

```
nn_high = NearestNeighbors(n_neighbors=k_nn+1, metric='cosine').fit(embeddings_norm)
neighbors_high = nn_high.kneighbors(embeddings_norm, return_distance=False)[:, 1:]

nn_low = NearestNeighbors(n_neighbors=k_nn+1, metric='euclidean').fit(embedding_2d)
neighbors_low = nn_low.kneighbors(embedding_2d, return_distance=False)[:, 1:]

overlaps = [
    len(set(neighbors_high[i]) & set(neighbors_low[i])) / k_nn
    for i in range(len(embeddings_norm))
]
mean_overlap = np.mean(overlaps)
print(f"Neighborhood Preservation (k={k_nn}): {mean_overlap:.4f}")
print(f"  {mean_overlap*100:.1f}% of top-{k_nn} neighbors preserved in 2D")

Neighborhood Preservation (k=10): 0.4193
  41.9% of top-10 neighbors preserved in 2D
  > 0.70 is strong for a recommender system
```

Step 13 — Cosine Similarity Distribution

This shows how similar the title embeddings are to each other overall.

Interpretation:

- **High mean similarity** (close to 1.0) → most playlist titles are semantically similar → UMAP may produce a blob with little structure
- **Low mean similarity** (close to 0.0) → titles are very diverse → expect clear clusters in the UMAP plot
- **Max close to 1.0** → some playlists have nearly identical title embeddings

```
sample_emb = embeddings_norm[:1000]
```